CASE 002 · DATABASE DETECTIVE

SQL 탐정단

e스포츠 결승전의 사라진 1위 기록

처음부터 다시 배우는 SQL 종합 추리 실습



SELECT → WHERE → GROUP BY → JOIN → 사건 해결

사건 발생! 우승 점수가 사라졌다

같은 날, 같은 선수의 점수가 갑자기 바뀌었다.

14:15 정상 기록

서준 선수의 결승 기록
9,800점

14:32 변경된 기록

서준 선수의 현재 기록
6,800점

미션: “어떤 계정이, 언제, 어떤 기기에서 수정했는가?”

오늘의 목표는 단순 암기가 아니다

실제로 단서를 찾으며 SQL 핵심 문법을 연결한다.

- ① 데이터를 찾기 — SELECT · WHERE · LIKE · ORDER BY · LIMIT
- ② 숫자를 읽기 — COUNT · GROUP BY · HAVING
- ③ 표를 합치기 — INNER JOIN · LEFT JOIN · IS NULL
- ④ 증거를 연결하기 — 다중 JOIN · 사건 보고서
- ⑤ 데이터를 관리하기 — CREATE · INSERT · UPDATE · DELETE

DB는 사건을 보관하는 증거 상자

한 개의 표는 한 종류의 정보를 맡는다.

DATABASE

여러 테이블을 모아
관리하는 데이터 공간

TABLE

학생 명단, 점수표처럼
같은 종류의 정보를 저장

ROW / COLUMN

행 = 선수 한 명의 정보
열 = 이름·팀·학년 같은 항목

핵심 질문: “선수 이름을 점수표에도 계속 반복해서 써야 할까?”

증거 장부는 네 개다

서로 다른 장부를 고유번호와 사건 시각으로 연결한다.

players

선수번호(PK) · 이름
팀 · 학년

scores

선수번호 · 현재점수
최종수정시각

score_history

선수번호 · 변경 전 점수
변경 후 점수 · 시각

access_logs

작업계정 · 대상선수
행동 · 시각 · 기기

기본키는 학생 이름표가 아닌 고유번호

이름이 같아도 번호가 다르면 다른 사람이다.

player_id (PK)	name	team
101	서준	블루
102	민아	레드
103	도윤	레드

player_id	points
101	6,800
102	8,700
103	8,200

연결 규칙: players.player_id = scores.player_id

모든 선수 명단 열람

SELECT는 볼 열, FROM은 읽을 표를 정한다.

```
SELECT name, team
FROM players;
```

- SELECT name, team : 이름과 팀만 보여줘
- FROM players : 선수 명단에서 찾아줘
- * 는 모든 열을 뜻한다.

필터: 레드팀만 찾으려면?

WHERE = 특정 조건에 맞는 행만 통과

```
SELECT name, team
FROM players
WHERE team = '레드';
```

미션 02 | 레드팀에는 누가 속할까?

비슷한 이름도 검색할 수 있다

%는 글자가 0개 이상 이어져도 된다는 뜻이다.

```
SELECT name
FROM players
WHERE name LIKE '서%';
```

- LIKE '서%' : '서'로 시작
- LIKE '%준' : '준'으로 끝남

결승전 TOP 3 만들기

정렬을 먼저, 개수 제한은 그 다음에 적용한다.

```
SELECT player_id, points
FROM scores
ORDER BY points DESC
LIMIT 3;
```

DESC: 큰 값 → 작은 값 | ASC: 작은 값 → 큰 값

한 장부를 통계로 바꿔보자

COUNT(*)는 행의 개수를 세는 함수

```
SELECT team, COUNT(*) AS 인원
FROM players
GROUP BY team;
```

GROUP BY team: 같은 팀을 한 묶음으로 모은다.

WHERE와 HAVING은 언제 다를까?

묶기 전의 조건과 묶은 후의 조건

WHERE

그룹을 만들기 전에
각 선수 정보를 거른다.

HAVING

GROUP BY가 만든
그룹의 집계 결과를 거른다.

```
SELECT team, COUNT(*) AS 인원
FROM players GROUP BY team
HAVING COUNT(*) >= 2;
```

JOIN이 필요한 이유

선수 이름은 명단에, 점수는 점수표에 따로 있다.

선수번호	이름
101	서준
102	민아
103	도윤



선수번호	점수
101	6,800
102	8,700
103	8,200

같은 선수번호를 찾아 두 장부를 합친다

INNER JOIN: 짝이 있는 행만

선수 명단과 현재 점수표를 연결

```
SELECT P.name, S.points
FROM players AS P
INNER JOIN scores AS S
    ON P.player_id = S.player_id;
```

선수 7명 중 점수가 등록된 선수는 6명 → 결과 6행

LEFT JOIN: 명단은 모두 남겨라

점수 기록이 없는 선수도 사건의 일부다.

```
SELECT P.name, S.points
FROM players P
LEFT JOIN scores S
    ON P.player_id = S.player_id;
```

미기록 선수의 점수는 NULL | NULL은 0과 다르다

미기록 선수만 찾아라

LEFT JOIN 이후 오른쪽 장부의 번호가 비었는지 확인

```
SELECT P.name
FROM players P
LEFT JOIN scores S
    ON P.player_id = S.player_id
WHERE S.score_id IS NULL;
```

주의: = NULL ✗ IS NULL ✓

새로운 단서: 점수 변경 이력

현재 점수만 보면 이전 기록이 보이지 않는다.

선수번호	변경 전	변경 후	시각
101	9,100	9,400	14:01
101	9,400	9,800	14:15
101	9,800	6,800	14:32

질문: 점수가 감소한 행을 WHERE로 찾으려면?

수상한 변경을 찾아라

같은 행의 '변경 전'과 '변경 후'를 비교한다.

```
SELECT player_id, old_points, new_points
FROM score_history
WHERE new_points < old_points;
```

핵심: 열끼리 비교도 가능하다.

접속 로그: 누가 기록을 바꿨을까?

작업한 계정, 대상 선수, 시각, 기기번호를 대조하자.

actor_id	target_id	action	log_time	device
103	101	LOGIN	14:28	PC-07
103	101	UPDATE_SCORE	14:32	PC-07
104	101	VIEW	14:33	PC-03

actor_id = 작업 계정 | target_id = 변경 대상 선수

같은 선수번호가 두 역할을 한다

players 테이블을 작업계정 A와 대상선수 P로 두 번 연결한다.

```
SELECT A.name AS 계정, P.name AS 대상선수
FROM access_logs L
JOIN players A ON A.player_id = L.actor_id
JOIN players P ON P.player_id = L.target_id
WHERE L.action = 'UPDATE_SCORE';
```

별명 A / P를 다르게 붙여 같은 표를 두 가지 역할로 활용

최종 검증: 기록 + 로그 연결

선수번호와 시각이 모두 같은지 교차 확인한다.

```
SELECT H.old_points, H.new_points, L.device
FROM score_history H
JOIN access_logs L
 ON H.player_id = L.target_id
AND H.changed_at = L.log_time
WHERE L.action = 'UPDATE_SCORE';
```

두 단서의 조건을 AND로 연결한다.

15개의 미션: 난이도에 따라 성장

학생용 ipynb에서 TODO를 완성하고 자동점검

LEVEL 1 / 복습

01 명단 보기 · 02 필터
03 LIKE · 04 정렬 · 05 TOP 3

LEVEL 2 / 분석

06 COUNT · 07 GROUP BY
08 HAVING · 09 INNER JOIN
10 LEFT JOIN

LEVEL 3 / 수사

11 정상점수 · 12 점수 감소
13 로그 계정 · 14 교차검증
15 최종 사건 보고

💡 실행하기 → 틀렸다면 수정 → 다시 실행 → 정답 확인

CREATE · INSERT · UPDATE · DELETE

수사에 사용할 연습용 메모 테이블만 수정한다.

CREATE

새 테이블 만들기

INSERT

새 메모 추가하기

UPDATE

기존 메모 수정하기

DELETE

필요 없는 메모 삭제하기

주의: WHERE 없는 UPDATE / DELETE는 여러 행을 변경할 수 있다.

사건 보고: 사실과 추측을 구분하자

SQL 결과를 그대로 읽고, 해석의 한계까지 설명한다.

데이터에서 확인되는 사실

14:32 / PC-07 / 도윤 계정
서준 9,800 → 6,800점

데이터만으로 단정할 수 없는 것

계정 사용자가 실제로
직접 수정했는지는 추가 확인 필요

정확한 탐정은 “기록된 계정”과 “실제 사람”을 구분한다.

최종 점검! 오늘 무엇을 할 수 있나요?

한 문장으로 설명하거나 SQL 한 줄로 증명해 보자.

- SELECT와 WHERE는 각각 어떤 역할일까?
- GROUP BY와 HAVING은 언제 사용할까?
- INNER JOIN과 LEFT JOIN은 무엇이 다를까?
- 왜 고유번호로 테이블을 연결할까?
- 사건에서 확실한 사실과 추정은 무엇일까?

사건 해결 완료!

기억할 것은 정답보다
증거를 연결하는 방법

김부장 × 혼이 SQL 탐정단
다음 사건에서도 JOIN으로 증명하자.

